CS781 Capstone Project: In-House Legal Intelligence System

Tejas Shrivastava
231091
tejass23@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

A legal team handling confidential judgments may need to run its models in-house. LEXWeave, a 95.23-million-parameter causal decoder, was trained on Indian statutes and judgments from the High Courts, the Supreme Court and tribunals. Phase 1 checkpoints supplied the starting weights for six downstream tasks. On 32 paired prompts, switching off the phrase mixer raised next-token loss by 4.460 nats per token. Switching off chunk memory cost just 0.016. Continued pretraining on a Tesla T4 reached a validation loss of 2.236, with mean local ROUGE-L of 0.1914 on 16 continuations. Retrieval exposed a different problem. With the dense encoder fixed, development Micro-F1@7 rose from 0.1167 for dense ranking to 0.3457 after combining it with full-text lexical ranking and excluding self-matches.

1 Competition Result

Codabench username: `t_231091` **Roll number:** 231091
GitHub repository: https://github.com/AlphaTS-01/CS781-LEGAL_LLM

Table 1: Reported best scores, leaderboard ranks and submission counts by phase.

Phase	Task	Primary metric	Score	Rank	Submissions
1	Legal continuation	BERTScore F1	0.8312*	78	7*
2	Legal entities	Strict Macro-F1	0.4957	84	6
3	Rhetorical roles	Macro-F1	0.4792	79	7
4	Judgment outcome	Macro-F1	0.6752	97	4*
5	Statute identification	Macro-F1	0.2685	111	6
6	Summarization	BERTScore F1	0.8276	101	5
7	Prior-case retrieval	Micro-F1@7	0.3538	86	6

Ranks were recorded on 9 November 2026. The reported total is 41 submissions across seven phases, with at least four in each phase.

2 Introduction and Problem Description

Consider a judgment that has to stay on a legal firm's own machines. What could a compact model do with it? Phase 1 starts with a passage and generates what might come next. Phase 2 locates and

types entity spans. Phase 3 takes the sentences in their given order and assigns each a rhetorical role. Two tasks need a decision about the whole case: a binary outcome in Phase 4 and a set of statute identifiers in Phase 5. The last two ask for quite different outputs—a summary in Phase 6 and seven distinct candidate cases for each query in Phase 7.

The foundation had to fit within 50–300 million parameters and use the supplied legal corpus. That still left choices about what information to keep. For an entity, its exact position matters. A case can have several statute labels. In retrieval the representation of one case has to be comparable with another's, while in summarization each new token depends on the judgment and the words generated so far.

Where would another hour of compute go? Longer pretraining was one possibility. Reading more of each judgment at inference or showing the task head more labeled examples were others. But even an encoder that recognizes a citation perfectly can waste one of seven retrieval slots by returning the query itself. That last explanation can be tested without training another encoder.

3 Data Analysis and Preprocessing

3.1 Foundation corpus and held-out data

The original cache contains about 9,450 training documents each from Acts and the High Courts. Count packed tokens instead and Acts outweigh the High Courts by a factor of 14.6 (Table 2). The seeded split covered all four official CS781 sources [1], and the manifest logged 29 duplicate removals. In a separate audit of the Acts file, four of 18,826 records were empty, while 74 ran to more than 100,000 whitespace-delimited words. The byte-level BPE tokenizer [2] was fitted on 30 million characters, with four special tokens for padding, beginning, end and unknown symbols.

Table 2: Original foundation archive. Documents come from the raw manifest and token counts from the packed manifest.

Source	Train docs	Train tokens	Val. docs	Val. tokens
Acts	9,454	84,085,550	90	668,912
High Courts	9,438	5,756,471	114	75,911
Supreme Court	9,437	47,979,953	97	441,438
Tribunals	9,445	40,567,664	107	600,798
Total	37,774	178,389,638	408	1,787,059

For continued pretraining, the notebook rebuilt the earlier selection so that it could exclude exact matches on canonical text. The new cache held 956,251,870 training tokens. Sampling was with replacement. Another 128 examples were set aside for continuation evaluation. Table 3 lists the separately recorded validation shards.

Table 3: Continued-pretraining validation cache, after exclusion of exact matches to the earlier selection.

Source	Documents	Tokens
Acts	134	1,129,014
High Courts	4,479	3,322,071
Supreme Court	528	2,365,909
Tribunals	1,677	3,410,921

3.2 Downstream supervision

Each task used its own official data. The NER folds give character offsets and types for entities in judgments. The Corporate Law and IT files give ordered sentences and their rhetorical roles. For the two case classifiers, a judgment has either one binary outcome or a set of statute IDs, depending on the task. Summarization pairs judgments with reference summaries. Retrieval has a different unit of evaluation: a query comes with a candidate pool and, on labeled splits, a gold set of relevant cases. A role prediction can be checked one sentence at a time. A retrieval score depends on which cases were in the pool for that split.

3.3 Task-specific input handling

For NER, the loader keeps each token's original start and end offsets. The first and last retained tokens of a decoded span give a half-open interval in the source string. The pipeline can trim whitespace and learn punctuation-trimming rules from training labels. For rhetorical roles, the supplied sentence boundaries stay in place even when several sentences are packed into one token-budgeted window. The outcome and statute classifiers pool selected document chunks. The summarizer takes text from the beginning, middle and end: 512 source tokens in total (instructions included) and up to 192 target tokens. Queries and candidates go through the same chunking procedure in the dense retriever.

4 Approach and Model Evolution

4.1 Foundation architecture

LEXWeave has twelve decoder blocks of width 768, a 16,384-entry vocabulary and room for 1,024 tokens. After RMS normalization [3], each block splits into three branches. Grouped-query causal attention [4] uses twelve query heads, four key/value heads and rotary positions [5]. Memory summarizes completed 32-token chunks. The phrase mixer runs convolutions at dilations 1, 4 and 16. A softmax chooses the mix for each token before the residual addition. In the first chunk there is nothing in memory yet, so that branch is masked. A gated feed-forward layer of intermediate width 2,048 supplies the second residual update. The exported weights contain 95,232,768 trainable parameters.

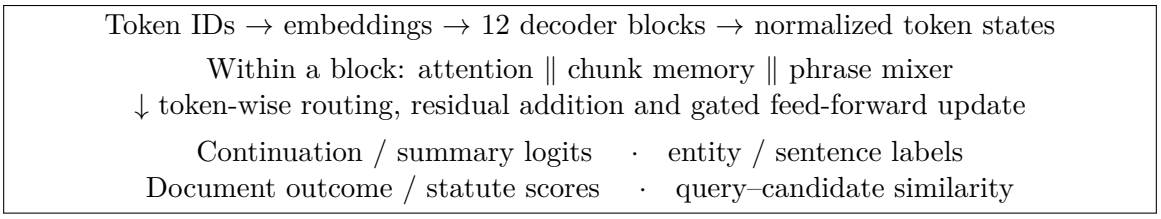


Figure 1: The foundation supplies token states to independently adapted task models.

4.2 Why use this foundation?

Attention alone [6] can connect a citation to its earlier mention inside the window. Why add two branches? The convolutions can pick up short phrases, while memory carries a summary of completed chunks. The router decides how much of each a token receives. All three fit within the cap at 95.23 million trainable parameters. Section 6.3 asks what happens when the trained decoder loses a branch at inference. Whether the branches paid for their training cost is a separate question, one that would take an attention-only run at the same budget.

4.3 Transfer from Phase 1 to the downstream tasks

LEXWeave supplies the token states in Phases 3–7. The best Phase 2 run took a different path through the foundation registry: an encoder–decoder with a CRF head and LoRA updates.

Phase 2 — Entity extraction. The primary configurations decode token tags with a conditional random field. An alternative head scores span endpoints with a biaffine transformation. It includes full and partial tuning, additive low-rank updates [7], multiplicative LoRMA updates [8], adapters and teacher–student distillation [9]. Strict scoring compares each predicted type and both character boundaries to the gold annotation.

Phase 3 — Rhetorical roles. The sentence head combines pooled token features with a learned domain embedding for Corporate Law or IT. A bidirectional LSTM can contextualize sentence representations before classification. Configurations include full tuning, LoRA, rank-stabilized LoRA [10], LoRMA, IA^3, soft prompts and distillation.

Phase 4 — Judgment outcomes. The default judgment model selects up to four 512-token chunks from the tail and uses attention to pool them. The document head pools token states within chunks, optionally adds a relative-position embedding, and aggregates valid chunks before predicting two logits. The implementation includes single-tail-chunk and first-chunk controls alongside its hierarchical model. It also supports gradual unfreezing, bias tuning and adaptive-rank updates.

Phase 5 — Statute identification. The model produces a score for each statute and calibrates the decision thresholds on labeled data. The budgeted Kaggle configuration encodes up to four 384-token windows per document. Its six configurations cover full and partial tuning, LoRMA+, bottleneck adapters [11], soft prompts and distillation.

Phase 6 — Summarization. The causal decoder receives source text followed by summary tokens. Training masks the source portion of the loss. The implementation compares LoRA, LoRMA+, two adapter placements, prompt tuning and logit distillation to a smaller student. Local scoring includes ROUGE-1 and ROUGE-L [12]. The competition uses BERTScore [13].

Phase 7 — Prior-case retrieval. A shared encoder pools up to two 256-token chunks from each case. It ranks the appropriate candidate pool and exports seven unique identifiers per query. A later retrieval experiment combined the LoRMA ranking with a full-text TF-IDF ranking and optionally removed self-matches. Our weighted version of reciprocal rank fusion [14] adds $1/(10+r_d)$

for a candidate in the dense top seven and $w/(10+r_l)$ for a candidate in the lexical top fifty. Ranks r_d and r_l start at one. Missing terms contribute zero. Table 5 records comparisons using the same trained dense model.

5 Experiments and Implementation Details

5.1 Compute and training protocol

The original foundation configuration specifies a peak learning rate of 3×10^{-4}, 300 warmup steps, a microbatch of four and four gradient accumulation steps. Continued pretraining restored the step-5,840 weights and tokenizer with a fresh optimizer. The executed notebook used one Tesla T4. A short benchmark recorded 2,165 loss tokens per second and 2.47 GiB of peak allocated GPU memory. The log records 48,996,269 target tokens at continued-training update 1,660, followed by an interruption and checkpoint save.

The Phase 5 configuration sets one epoch per method. Full tuning receives up to 180 minutes and 15,000 documents. Partial tuning, LoRMA+ and adapters receive 90 minutes and 7,000 documents each. Prompt tuning and distillation receive 45 minutes and 3,500 documents each. These are configured ceilings. Selection and calibration use separate development samples of up to 256 and 512 documents. The schedules differ in both compute and data exposure.

The local integration tests for Phases 5–7 exercised training and export on synthetic examples. They also checked compatibility with the real foundation checkpoint. The Phase 5 partial-tuning run exported its test predictions separately. In Phase 6, a Kaggle run encountered an invalid summary target during loading. The revised loader skips missing or blank training targets and retains the unlabeled test rows.

5.2 How much does each adaptation method update?

The rhetorical-role model provides a concrete reason to compare adaptation methods under a GPU budget. Its training harness loaded the real foundation checkpoint and counted parameters for each assembled model. Full tuning updated 96.32 million parameters. LoRMA updated 3.74 million, or 3.78% of its model, while soft prompts and their trainable task components updated 1.10 million, or 1.14%. This reduces the number of gradients and optimizer states that must be stored. The backbone still participates in the forward pass. Distillation addresses a different cost: the harness built a 55.00-million-parameter student, reducing the size of the model needed after training. These counts come from instantiated models; the synthetic harness cannot establish their ranking on legal cases.

5.3 Controlled evaluations

The trained router mixes three sources of information in every decoder block. To measure dependence on each branch, we ran the step-5,840 checkpoint in evaluation mode and masked selected branches before the routing softmax. This redistributed weight among the active branches. All four conditions used identical model weights and target tokens.

We sampled 32 of the 1,000 official Phase 1 prompt IDs after lexicographic sorting, using Python's random sampler with seed 42. All prompts met the 129-token minimum. We took the final 129 tokens from each visible prompt and scored its last 32 tokens by teacher forcing. This yielded 1,024 next-token predictions per condition. The diagnostic uses only supplied prompt text and requires no hidden continuation references. For Phase 7, we held the dense checkpoint fixed

across six development rankings. Each ranking used the same 118 queries and candidate pool. The comparisons varied self-match filtering, lexical backfill and the weight assigned to the full-text ranking (Table 5).

6 Results

6.1 Which submissions led under the available budget?

Table 4 records the variation within each task. In Phase 3, the LoRA submission exceeded the LoRMA submission by 0.0621 Macro-F1. In Phase 6 their order reversed, with only 0.0007 BERTScore F1 between them. This gives a practical reason to retain both adaptation methods when selecting a model for a new task.

Table 4: Codabench submissions by phase. Bold marks the highest reported primary score.

Submission	Score
Phase 1 — Legal continuation	
v3 submission	**0.8312***
v2 submission	0.8307
LEXWeave decode v2	0.8300
Anchor GPT 150M v2	0.8286
Relay LSTM 200M	0.8250
Anchor GPT 150M v1	0.8240
Bridge 200M	0.8067
Phase 2 — Legal entities	
E3 LoRA	**0.4957**
E2 LoRMA	0.4956
final	0.4858
E5 partial	0.4853
E6 KD	0.4629
E1 full	0.4486
Phase 3 — Rhetorical roles	
LoRA	**0.4792**
KD	0.4505
rsLoRA	0.4442
`submission.zip`	0.4306
prompt	0.4199
IA^3	0.4185
LoRMA	0.4171

Submission	Score
Phase 4 — Judgment outcome	
E0	**0.6752**
E3	0.6704
E1	0.4940
Fourth submission	—*
Phase 5 — Statute identification	
Full	**0.2685**
LoRMA	0.2298
adapter	0.2218
prompt	0.1622
partial	0.1488
KD	0.1307
Phase 6 — Summarization	
LoRMA	**0.8276**
LoRA	0.8269
Pfeiffer adapter	0.8263
Houlsby adapter	0.8253
prompt	0.8162
Phase 7 — Prior-case retrieval	
LoRMA confirmed	**0.3538**
prompt hybrid	0.3473
partial hybrid	0.3473
KD hybrid	0.3467
IA^3 hybrid	0.3467
full hybrid	0.3455

Phase 5 shows how the time allocation affects interpretation. Full fine-tuning had the largest allowance and led LoRMA by 0.0387 Macro-F1. Full and LoRMA were capped at 180 and 90 minutes respectively, and their document caps also differed. The result identifies the best observed submission under that allocation. Estimating the return from an additional GPU-hour requires evaluating each method at successive budgets using the same development cases. In Phase 7 all six finished submissions fell within 0.0083 Micro-F1@7 of one another.

6.2 Foundation and retrieval diagnostics

On 12 local continuations, the original checkpoint recorded ROUGE-L F1 of 0.1990 and SacreBLEU BLEU-4 [15] of 5.2653. At continued-training step 1,500 the notebook restored a checkpoint with 44,292,789 target tokens and validation loss 2.2361. Its 16-example continuation evaluation recorded mean local ROUGE-L of 0.1914. Across the 16 logged examples, the median was 0.1875 and the range was 0.1172 to 0.2578. The two checkpoint means therefore answer different sample-specific questions. Measuring the change due to continued training requires scoring both checkpoints on identical prompts with the same decoding settings.

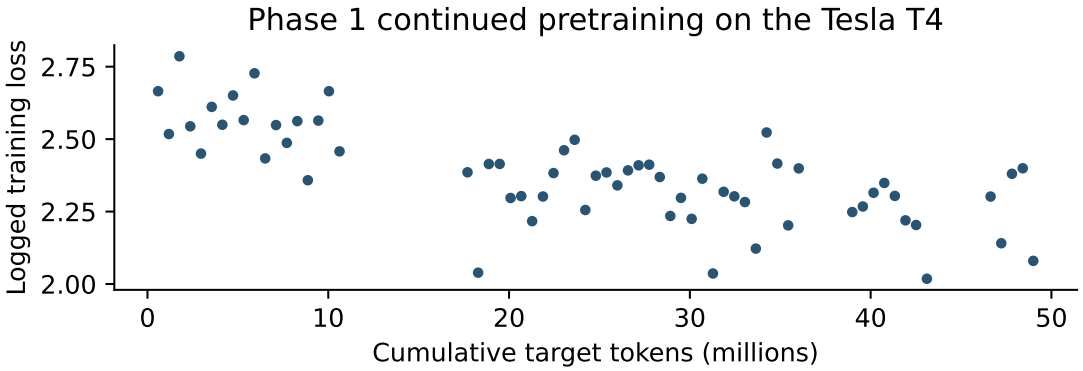


Table 5: Prior-case retrieval on 118 development queries. The dense model had completed 216 training steps. Weight refers to the lexical contribution in rank fusion.

Ranking	Self-match handling	Micro-F1@7
Dense baseline	Retained	0.1167
Full-text TF-IDF	Retained	0.3335
Hybrid, weight 2	Retained	0.3257
Dense, lexical backfill	Excluded	0.2112
Hybrid, weight 1	Excluded	0.2890
Hybrid, weight 2	Excluded	0.3457

6.3 Which branches does the trained decoder use?

Table 6: Paired inference-time branch masks. Loss is mean negative log-likelihood in nats per token. The final column counts prompts with higher loss than the intact model.

Active branches	Loss	Change	Prompts worse
Attention, memory, phrase mixer	2.999	–	–
Attention and phrase mixer	3.015	+0.016	23/32
Attention and memory	7.459	+4.460	32/32
Attention alone	8.472	+5.473	32/32

The effect of masking memory depended on which other branches remained active. With the phrase mixer present, masking memory cost 0.016 nats per token on average. With the phrase

mixer already masked, removing memory raised loss on all 32 prompts, by 1.013 nats per token on average (range 0.321–1.518). Memory therefore supplied information the trained decoder could use even on short inputs when it lost the phrase branch.

The trained network depended much more heavily on the phrase mixer for these short passages. Removing memory alone increased loss by 0.016 nats per token, with a 95% paired prompt-bootstrap interval of $[0.003, 0.029]$ (10,000 resamples). The 128-token input spans only four memory chunks. A longer-context evaluation would give memory more preceding chunks to use. Training a separate attention-only model at matched compute would answer a further question: how well attention can compensate when it learns without the other branches from the outset.

7 Error Analysis and Discussion

7.1 Why the retriever ranked its query first

The dense baseline placed the query first for all 118 development queries and all 237 test queries. None of the 118 development gold sets included the query itself. Because queries and candidates pass through the same encoder, the same text can produce identical vectors. The ID audit gives an actionable explanation for one wasted slot per query. Filtering that ID and backfilling raised Dev Micro-F1@7 from 0.1167 to 0.2112 with the checkpoint held fixed.

A second decision concerns the text available to the scorer. The dense encoder saw an average of 7.8% of query tokens and 12.9% of candidate tokens on Dev. Full-text TF-IDF alone reached 0.3335 with self-matches retained. Under the same self-filtering rule, increasing the lexical fusion weight from one to two raised F1 from 0.2890 to 0.3457. The lexical system changes both the matching function and the amount of text read. A TF-IDF run restricted to the dense encoder's exact spans would separate the coverage effect from the effect of lexical matching. A self-filtered full-text lexical baseline would also test how much the dense scores add to the strongest recorded hybrid.

7.2 Disagreement between submitted label sets

Entity recognition. The LoRA and LoRMA submissions scored 0.4957 and 0.4956 strict Macro-F1. Across the 35 test judgments, LoRA predicted 4,054 distinct typed spans and LoRMA predicted 3,948. Only 2,855 matched on document, start, end and entity type. Counting each span equally, their pairwise agreement was $2(2{,}855)/(4{,}054+3{,}948) = 0.714$. The leaderboard scores obscure this disagreement. Reviewing the unmatched spans on labeled cases would show how often the methods differ on the type, the boundary or the presence of an entity.

Statute identification. Full tuning and LoRMA predicted identical statute sets for only 94 of 13,019 test cases. The median case received six statute IDs from full tuning and seven from LoRMA. Full tuning used all 100 IDs across the test set, with statute 5 appearing in 6,363 cases. These counts show how widely the decisions changed, but the test labels are hidden. To locate the source of the 0.0387 Macro-F1 gap, the next comparison would report per-statute precision and recall on the same labeled cases, first with a shared cutoff and then with each model's calibrated cutoffs.

7.3 Fallback continuations and incomplete summaries

Legal continuation. One interrupted Phase 1 export used 401 primary generations, 593 fallback entries, two trimmed fallback entries and four entries repaired using prompt text. The v2 submission

used an earlier merge. Comparing its 1,000 records by prompt ID with the decode-v2 fallback archive found 255 changed records. Its 0.0007 BERTScore F1 advantage therefore came from a submission in which 745 records were unchanged. A fair decoding comparison would preserve the provenance of each prediction and score the same IDs from each model.

Summarization. In case 4963, the submitted LoRMA summary introduces the Revenue's argument and ends mid-clause at "the assessee could not". A simple export check found that 94 of its 100 summaries ended without a period, question mark or exclamation mark, as did 98 of the 100 LoRA summaries. This check flags endings for inspection and can also flag complete sentences followed by a closing quote. For case 4963 the missing clause is visible in the text itself. Logging whether decoding stopped at the length cap or emitted an end token would help distinguish the two causes before changing the output budget.

In Phase 6, a missing or blank target stopped data loading before the model could learn from that record. The loader fix restores training on usable pairs. Its regression test verifies handling of malformed records; summary quality requires a separate evaluation on real cases.

7.4 Limits of the metrics and training budgets

The compute budgets also affect interpretation. In Phase 5, full tuning has a larger document allowance than prompt tuning. A difference in their scores could reflect data exposure as well as which parameters were updated. In Phase 6, overlap metrics can reward reuse of the reference wording even when a summary assigns a ruling to the wrong party. Checking the parties, court and disposition against the source would address that limitation directly.

7.5 Follow-up comparisons

For NER, passing gold-aligned token tags through the character-span decoder would measure the recall lost during span reconstruction. A gold entity whose endpoint falls inside a token can remain unreachable even with a correct token prediction. Comparing strict recall before and after boundary trimming on held-out documents would quantify that loss. Remaining errors can then be split into wrong boundaries and wrong entity types.

For rhetorical roles, sentence-only classification provides a baseline for the document-context model. A sentence quoting a prior ruling may resemble the current court's decision when read alone. Scoring both models on the same sentences, separately for Corporate Law and IT, would identify which role confusions change with context. For judgment prediction, the corresponding question is where the outcome evidence lies. Comparing four opening chunks with four closing chunks at the same token budget would isolate location. The single-tail-chunk comparison changes both coverage and pooling, so it cannot by itself identify a benefit from hierarchy.

For statutes, the simplest baseline applies a threshold of 0.5 to every label. A rare statute may receive useful scores below that cutoff. The implemented rule searches 37 cutoffs from 0.05 to 0.95. A statute receives a separate cutoff only when the calibration subset contains at least five positives and five negatives for it. Otherwise it inherits the global cutoff. This prevents a single rare positive from determining that statute's threshold. The direct test is to calibrate thresholds on one labeled subset and apply both decision rules to the same untouched cases using the same saved probabilities. For summarization, an opening-only input provides a baseline for the three-span input. With the source-token budget fixed, one can check whether a reference disposition near the end of the case appears in the generated summary and whether the summary attributes that disposition to the correct court.

8 Conclusion

The deadline forced a choice between training the shared foundation longer and completing the downstream tasks. My hypothesis is that additional Phase 1 training would improve transfer and narrow the gap to the strongest systems. The continued run logged 49 million target tokens against a 956-million-token cache. The first scaling experiment would evaluate the original checkpoint and saved continuation checkpoints on one fixed set of prompts. Each checkpoint would then initialize the same downstream recipe with identical labeled splits, update counts and decoding rules. I would report the change in each task's primary metric alongside GPU-hours. Consistent gains across all six tasks would support a shared pretraining bottleneck. A flat result on one task would direct the next experiment toward its own input representation or decision rule. Phase 7 already provides an instance where a ranking change improved retrieval without another training step.

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Claude x / Grok / Copilot / other).

Answer: GPT-6.1 Sol (OpenAI) and Claude Sonnet 5 (Anthropic). **Q2.** For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Explained span decoding and lexical retrieval.
Ideation / brainstorming at a high level	Yes	Suggested comparisons using the same checkpoint.
Debugging small ancillary snippets	Yes	Helped diagnose missing summary targets during data loading.
Plotting / visualization code	Yes	Plotted the recorded Phase 1 training loss.
Grammar / writing style of the report	Yes	Revised wording in the report and declaration.
Report structure / section wording	Yes	I used an LLM to turn a bulleted outline of the work and results from all seven phases into coherent, grammatically correct prose.
Designing the model architecture	No	
Hyperparameter search / tuning	No	
Analysis of results / conclusions	No	
Writing the core training / inference code	No	
Other (specify)	No	

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer: Yes. I used the CS781 specification for the task requirements and the official dataset [1] for the legal texts and labels. The report cites LoRA and LoRMA for the two adaptation methods [7, 8], ROUGE and BERTScore for generation evaluation [12, 13], and reciprocal rank fusion for combining retrieval results [14]. The Phase 7 notebook uses scikit-learn's `TfidfVectorizer` to rank full case texts. A CS781 capstone report informed the choice to compare NER predictions span by span. I used my archived submissions for that comparison.

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them. Answer: I wanted to know how much the six downstream tasks depended on the training budget for LEXWeave, the 95.23-million-parameter decoder in Phase 1. The continued run processed about 49 million target tokens on a Tesla T4, so the first comparison I would make is between earlier and later checkpoints on the same prompts. One problem with using the leaderboard for that comparison was that a generation run stopped after producing only 401 of the 1,000 primary continuations. Fallback and repaired outputs filled the remaining entries. I would compare the same prompt IDs across decoding runs before attributing a score change to the model.

A second failure was easier to trace. The dense retriever returned the query itself in first place for all 118 development queries, costing one of the seven slots each time. Removing that ID and backfilling from the lexical ranking raised Micro-F1@7 from 0.1167 to 0.2112 with the checkpoint held fixed. Giving the full-text lexical ranking more weight in the filtered hybrid brought the score to 0.3457. That comparison is why I would fix the candidate rule and the amount of text each method sees before spending more GPU time on the retriever.

References

[1] Exploration Lab, "CS-781 Capstone Dataset." https://huggingface.co/datasets/Exploration-Lab/CS-781-Capstone.

[2] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," *Proceedings of ACL*, 2016.

[3] B. Zhang and R. Sennrich, "Root mean square layer normalization," *Advances in Neural Information Processing Systems*, 2019.

[4] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebron, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," *Proceedings of EMNLP*, 2023.

[5] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv:2104.09864*, 2021.

[6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[7] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," *International Conference on Learning Representations*, 2022.

[8] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," *Findings of ACL*, 2025.

[9] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv:1503.02531*, 2015.

[10] D. Kalajdzievski, "A rank stabilization scaling factor for fine-tuning with LoRA," *arXiv:2312.03732*, 2023.

[11] N. Houlsby, A. Giurgiu, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for NLP," *Proceedings of ICML*, 2019.

[12] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, pp. 74–81, 2004.

[13] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *International Conference on Learning Representations*, 2020.

[14] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," *Proceedings of SIGIR*, 2009.

[15] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in *Proceedings of ACL*, pp. 311–318, 2002.